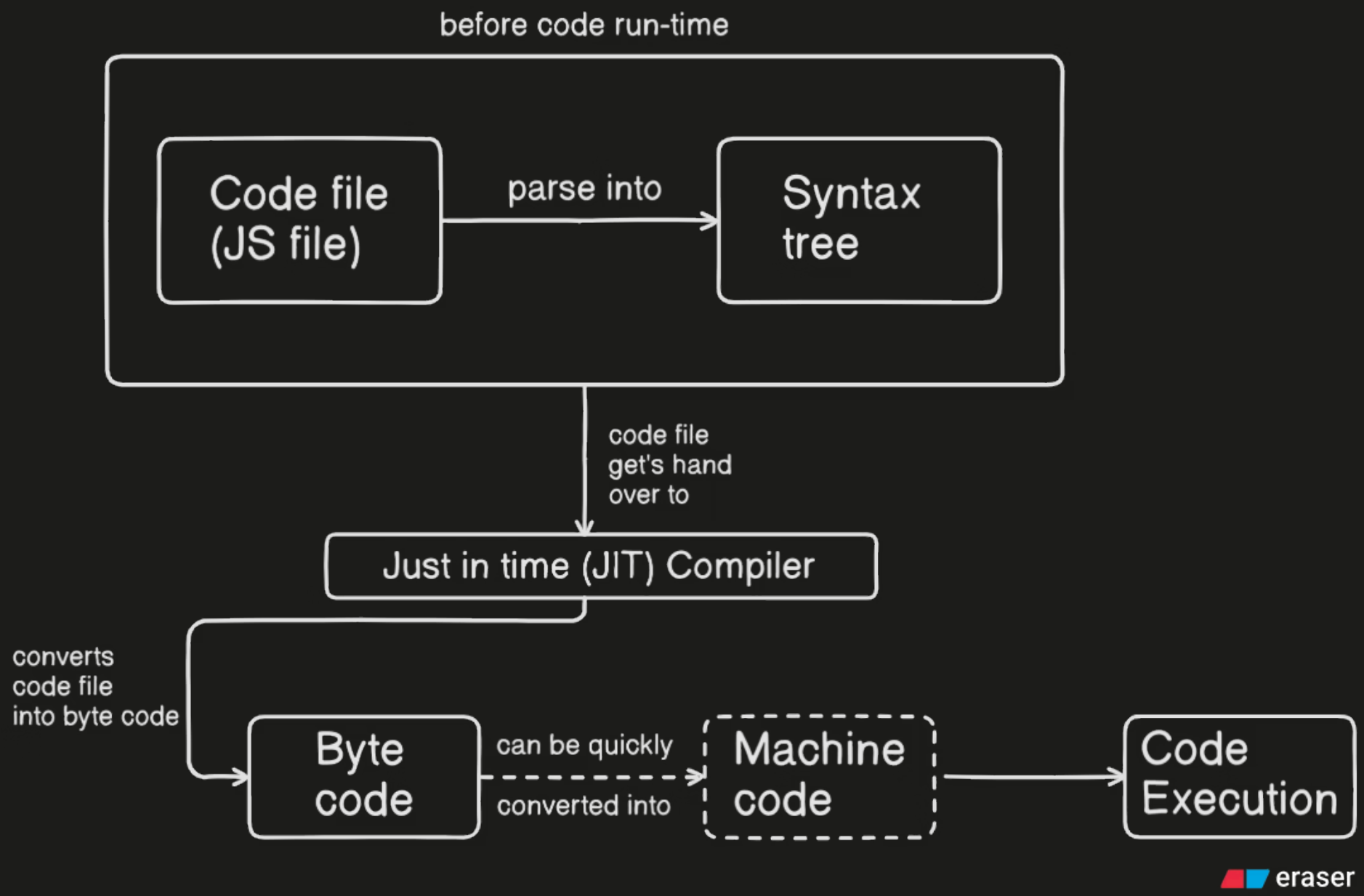
Basics of JavaScript

Lec-2

How JavaScript Executes behind the scene



- Initially your **Js code file** get's parsed into a **syntax tree** by **node or browser's V8 engine.**
- The **role of syntax Tree** is to **check and verify syntax of your code.**
- After that the **code file get's handed over** to **Just In Time (JIT) compiler (JIT is an inbuilt part of Node js)**.
- **JIT converts code file into byte code.**
- **If required byte code** can **convert** code file into **machine code**.
- Then the code finally get's **executed**.

Operators in JavaScript

- In **JavaScript** there are following types of operators:

1. **Arithmetic Operators**
2. **Assignment Operators**
3. **Comparison Operators**
4. **Logical Operators**

Arithmetic Operators

- They're used to **perform arithmetic operations on numbers**.

Operator	Name	Example
+	Addition	x + y
-	Subtraction	x - y
*	Multiplication	x * y
**	Exponentiation	x **y
/	Division	x / y
%	Modulus	x % y
++	Increment	x ++
- -	Decrement	x - -

- **Eg:**

```
let var1 = 20
let var2 = 50

console.log(var1 + var2)
70

console.log(var1 - var2)
-30

console.log(var1 * var2)
1000

let var3 = 5
let var4 = 2

console.log(var3 ** var4)
25

console.log(var3 / var4)
2.5

console.log(var3 % var4)
1

console.log(var3++)
6

console.log(var4 --)
1
```

Assignment Operators

- Assignment operators **assign values to JavaScript variables**.

Operator	Example	Same as
=	x = y	x = y
+ =	x += y	x = x + y
- =	x -= y	x = x - y
* =	x *= y	x = x * y
** =	x **= y	x = x ** y
/ =	x /= y	x = x / y
% =	x %= y	x = x % y

- **Eg:**

```
let x = 10
let y = 5

console.log(x)
10
console.log(y)
5
console.log(x+=y)
15
console.log(x-=y)
10
console.log(x*=y)
50
console.log(x/=y)
10
console.log(x%=y)
0
```

Comparison Operators

- Comparison operators are used to **compare two values**.
- Comparison operators always return **true** or **false.**

Operator	Description	Example
= =	equal to	x == 5
= = =	equal value and equal type	x === 5
! =	not equal	x != 5
! = =	not equal value or not equal type	x !== 5
>	greater than	x > 5
<	less than	x < 5
> =	greater than or equal to	x >= 5
< =	less than or equal to	x <= 5

- **Eg:**

```
let var1 = 'hello'
let var2 = 'hi'

console.log(var1 == var2)
false

console.log(var1 === var2)
false

var2 = 'hello'

console.log(var1 === var2)
true

console.log(var1 != var2)
false

console.log(var1 !== var2)
false

let num1 = 15
let num2 = 13

console.log(num2 > num1)
false

console.log(num2 < num1)
true

num1 = 13

console.log(num2 >= num1)
true

console.log(num2 <= num1)
true
```

Logical Operators

- Logical Operators are used to **combine Boolean expressions**.
- They can be used to modify results of comparison.

Operator	Name	Example
&&	**AND**	(x < 10 && y > 1) is true
||	**OR**	(x === 5 || y === 5) is false
!	**NOT**	!(x === y) is true

Eg:

```
let var1 = 50;
let var2 = 100;

console.log(var1 < var2 && var1 != var2)
true
console.log(var1 < var2 && var1 !== var2)
true

var2 = 'hundred';
console.log(var1 < var2 && var1 != var2)
false
console.log(var1 < var2 && var1 !== var2)
false

var2 = 100;
console.log(var1 < var2 || var1 != var2)
true
```

```
let s = 17
let k = 17
console.log(!(s === k))
false
```

Null Coalescing Operator (??)

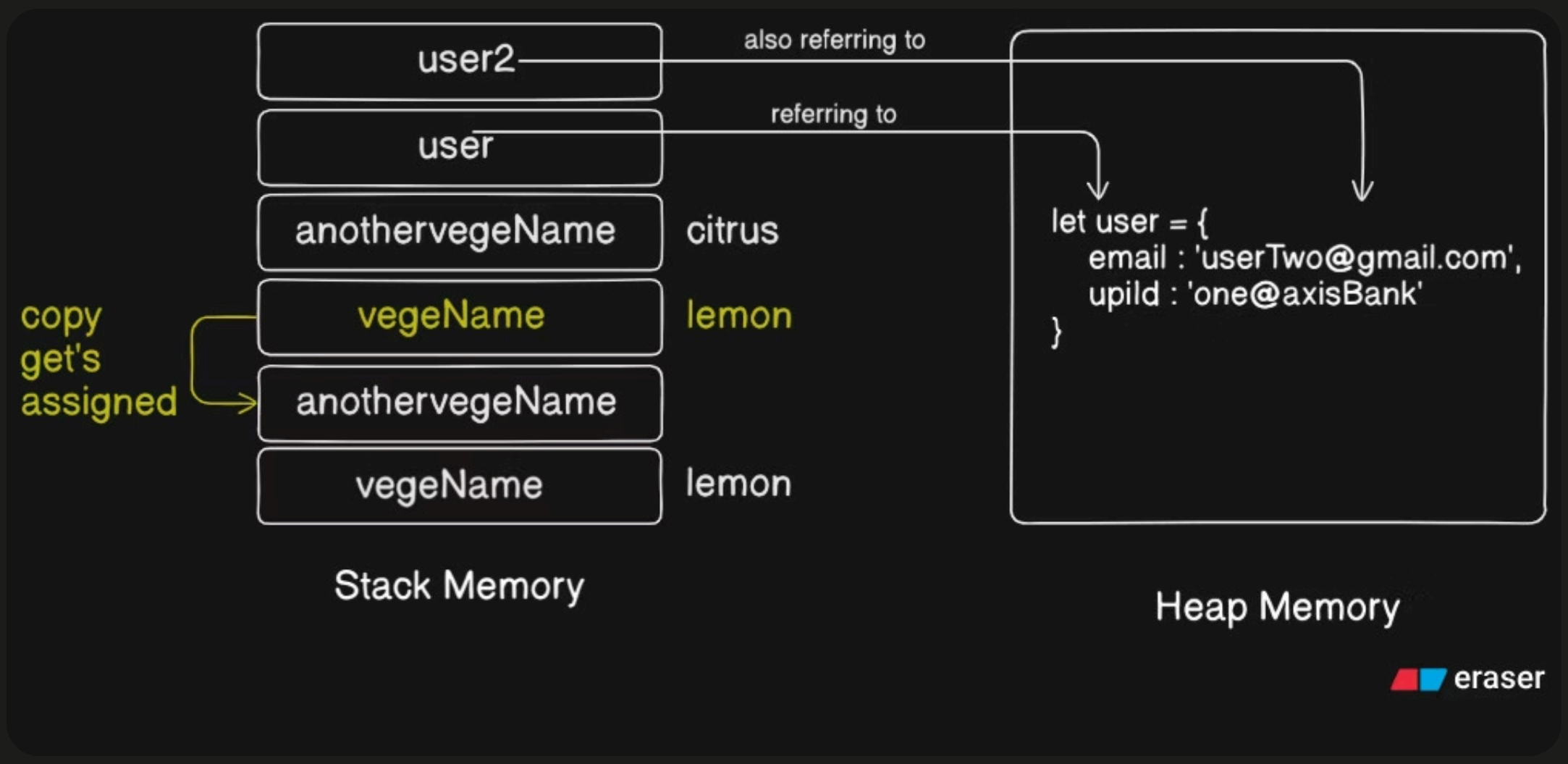
- The `??` operator returns the right operand when the left operand is **null** or **undefined.**
- **Otherwise it returns the left operand**.

Eg:

```
let test1 = 'testVar1'
let test2 = 'testVar2'

let result = test1 ?? test2
console.log(result)
testVar1

test1 = null
result = test1 ?? test2
console.log(result)
testVar2


test1 = undefined
result = test1 ?? test2
console.log(result)
testVar2
```

Stack Memory and Heap Memory

Stack Memory	Heap Memory
Only **primitive datatypes** are stored in **stack memory.**	Only **non-primitive datatypes** are stored in **heap memory.**
When any data is stored into **stack memory** we get **copy of that data.**	When any data is stored into **heap memory** we get **reference of that data**



Eg:

```
"use strict"
let vegeName = 'lemon'
let anothervegeName = vegeName

console.log(anothervegeName);
anothervegeName = 'citrus'

console.log(anothervegeName);

console.log(vegeName);


let user = {
    email : 'userOne@gmail.com',
    upiId : 'one@axisBank'
}

let user2 = user
user2.email = 'userTwo@gmail.com'

console.log(user2.email);
console.log(user.email);
```